Suicidal Text Detection in Social Media (Reddit) Posts

1. Udbhav Srivastava (532008960)
2. Gunjan Joshi (333000259)
3. K. Sreesh Reddy (433004917)

Team Number: 5

Table of Contents

ABSTRACT 3

INTRODUCTION 4

DATASET 4

METHODOLOGY 6

Preprocessing *6*

Cleaning the Data *7*

Exploratory Data Analysis *9*

Word Cloud 9
Average Text Length Distribution 10
Frequent Bigrams 10

Word Embeddings *11*

Word2Vec 11

Model Building *12*

1. **Logistic Regression** 13
2. **CNN (Convolutional Neural Network)** 14
3. **LSTM (Long Short-Term Memory Neural Network)** 16

Transformers Architecture 18

4. **BERT** 24
5. **ELECTRA** 25

..... 27

..... 27

RESULTS 27

CONCLUSION 28

REFERENCES 29

TEAM MEMBER CONTRIBUTIONS

Udbhav Srivastava

In my contribution to this project, I performed Exploratory Data Analysis (EDA) to understand the language used in suicidal and non-suicidal social media posts. This analysis involved looking at how often words appeared, the distribution of post lengths, and bigrams (two consecutive words that provide context). To capture the deeper meaning behind the words, I leveraged Bidirectional Encoder Representations from Transformers (BERT), a powerful Natural Language Processing (NLP) technique. I fine-tuned the pre-trained BERT model on our specific dataset of social media posts. This fine-tuning process involved adding a custom layer on top of the pre-trained BERT architecture and training it to distinguish between suicidal and non-suicidal language. Overall, my focus was on data exploration, feature engineering, and fine-tuning a BERT model to achieve high accuracy in classifying suicidal ideation in social media posts.

Gunjan Joshi

My key contributions to this project focused on preparing the dataset and implementing advanced language models. I cleaned and preprocessed the messy data, ensuring its readiness for exploratory analysis and subsequent model training. For the classification task, I successfully implemented the ELECTRA model from Hugging Face Transformers to distinguish between suicidal and non-suicidal Reddit posts. Additionally, I worked on developing a prototype LLM mental health chatbot using the QLORA technique and Meta's LLAMA 2 model. This prototype exhibits significant potential in providing therapeutic guidance to patients, illustrating the power of LLMs in the mental health domain.

K. Sreesh Reddy

In this project, I helped in the preprocessing of the dataset to generate data suitable to be passed through machine learning models, and implementation of key ML and DL models, including Logistic Regression, CNN, and LSTM. I followed a structured flow, transforming raw text into suitable inputs for each model through various preprocessing steps such as tokenization, embedding creation, and data cleaning. I also developed a custom Word2Vec pre-training model, enriching the embedding layer's representations and improving the overall performance of the models. This comprehensive approach ensured that each model effectively captured the nuances of the textual data, facilitating accurate detection of suicidal intent.

ABSTRACT

According to a News Release published by KFF, an independent source for health policy research, roughly 1 in 5 adolescents report symptoms of anxiety and depression. In particular, 14.7% of the US population aged 12 to 17 had at least one major depressive episode. Major depression is the most prevalent mental condition in the United States and is considered to be the strongest risk factor for suicidal behavior. Social media sites such as Reddit have become a common medium for sharing personal challenges, especially those connected to mental health. Early diagnosis of suicide intent among youngsters is critical for intervention and therapy to prevent irreparable consequences. Therefore, it is imperative to create effective big data systems that use cutting-edge Natural Language Processing (NLP) methods to extract suicidal intent from social media data. The project aims to analyze posts with suicidal intentions and build a predictive model to detect posts that do contain suicidal ideation.

INTRODUCTION

The mental health crisis among adolescents has reached alarming proportions, with 1 in 5 reporting symptoms of anxiety and depression. According to the National Institute of Mental Health, 14.7% of the US population aged 12 to 17 experienced at least one major depressive episode in 2021[1]. This highlights the urgency of addressing mental health issues in this vulnerable demographic, particularly given the strong correlation between major depression and suicidal behavior.

In the digital age, social media platforms such as Reddit have become prevalent spaces for individuals to express personal struggles, including those related to mental health. This shift to online communication presents a dual opportunity: on one hand, it allows for broader, candid discussions about mental well-being, yet on the other, it emphasizes the need for systems capable of monitoring these conversations to identify at-risk individuals early.

This project addresses this pressing issue by leveraging cutting-edge Natural Language Processing (NLP) techniques to analyze social media posts for signs of suicidal ideation. By developing and refining predictive models, we aim to create an effective big data system capable of early detection of posts containing suicidal intent. Such timely intervention can play a crucial role in preventing irreparable consequences, ultimately supporting adolescents in navigating mental health challenges with adequate therapy and support.

DATASET

The dataset[2] used for the project was collected from Kaggle, which contained posts from two subreddits present on the social media site Reddit; r/SuicideWatch and r/depression. Postings on "SuicideWatch" from its founding on December 16, 2008, to January 2, 2021, were all gathered, while postings on "depression" were collected between January 1, 2009, and January 2, 2021. Posts from the teenagers' subreddit are categorized as non-suicidal, while all posts gathered from SuicideWatch are labeled as suicide.

	Unnamed: 0	text	class
0	2	Ex Wife Threatening SuicideRecently I left my ...	suicide
1	3	Am I weird I don't get affected by compliments...	non-suicide
2	4	Finally 2020 is almost over... So I can never ...	non-suicide
3	8	i need helpjust help me im crying so hard	suicide
4	9	I'm so lostHello, my name is Adam (16) and I'v...	suicide

Figure 1: Snippet of Dataset

The dataset contains 232074 rows and 2 columns. The "text" column indicates the content of the post, and the "class" column indicates what the post is labeled as. "Class" has only 2 unique values: 'suicide' and 'non-suicide'.

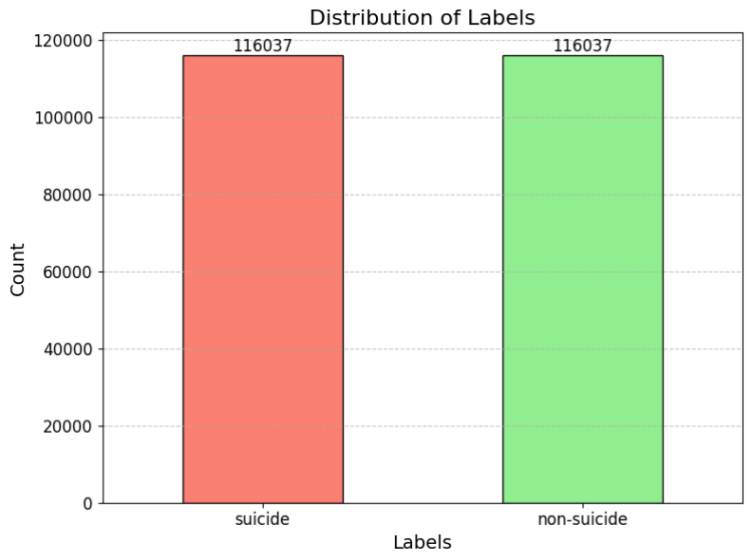

Figure 2: Distribution of Class Labels

The distribution of the classes is equal, where each class contains 116,037 rows or 50% of the dataset.

METHODOLOGY

Preprocessing

Since posts usually uploaded on social media are not structured in nature, they need to go through some sort of preprocessing so that they're in a suitable format that can be inputted into models built later.

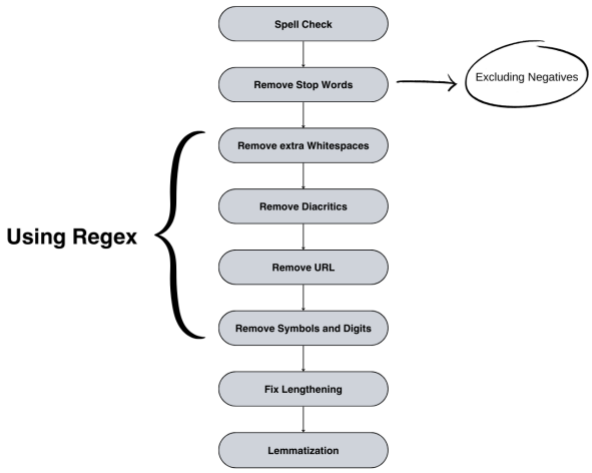

Figure 3: Flow of Preprocessing text

Spell Check: This function uses the SymSpell[3] library to correct spelling mistakes in a given text. It provides a suggestion based on a lookup process, considering a maximum edit distance of 2, and returns the corrected text or the original text if no correction is needed.

Remove Stop Words: This segment excludes some essential words from the stopwords list, ensuring that meaningful words like "not," "never," and "but" are retained for accurate text processing and analysis, particularly in sentiment analysis.

Remove extra White Spaces: This function trims any leading or trailing whitespace from the text and replaces multiple internal spaces with a single space, ensuring the text is clean and ready for further processing.

Remove Diacritics: This function removes accented characters from text, such as those found in words like "café," by converting them to their unaccented equivalents, improving the text's consistency and making it easier to process.

Remove URL: This function removes any URLs from the text using a regular expression, cleaning the text by eliminating unnecessary links and potential noise.

Remove Symbols and Digits: This function eliminates all characters from the text except for

alphabetic characters and spaces, ensuring the text only contains words, makes it suitable for further language processing tasks.

Fix Lengthening of Words: Word Lengthening occurs when characters are repeated in an incorrect manner wrongly in terms of the syntax of the word. Since words in English have a maximum of two characters repeated, the additional characters must be ripped off. For example, “finallllllly” is a lengthened version of ‘finally’. This is done using Regular Expressions in Python.

Lemmatization: This function converts tokens to their base forms, known as lemmas, during the text preprocessing process. It uses the Spacy NLP library to achieve this, replacing words with their lemma unless the token is a pronoun. This process helps standardize the text, making it easier to analyze and compare.

	text	class	cleaned_text
1	Am I weird I don't get affected by compliments...	non-suicide	weird not affect compliment come know girl but...
2	Finally 2020 is almost over... So I can never ...	non-suicide	finally never hear bad year swear fucking god ...
3	i need helpjust help me im crying so hard	suicide	need help help cry hard
4	It ends tonight.I can’t do it anymore. \nI quit.	suicide	end tonight not anymore quit
5	I took the rest of my sleeping pills and my pa...	suicide	take rest sleeping pill painkiller not wait en...

Figure 4: Dataset with preprocessed text

Cleaning the Data

Removing Redundant Words: After conducting initial data exploration, we analyzed the frequency of each word in the preprocessed posts. We discovered that the word "filler" appeared 55,442 times among the 30 most frequent words, which seemed anomalous. Upon further investigation, we realized that "filler" was noise captured during data collection, lacking meaningful content. Consequently, it was removed from all posts.

Deleting Empty Rows: Following text preparation and the elimination of redundant words, there were about sixty blank text fields devoid of any text. These matching rows were removed.

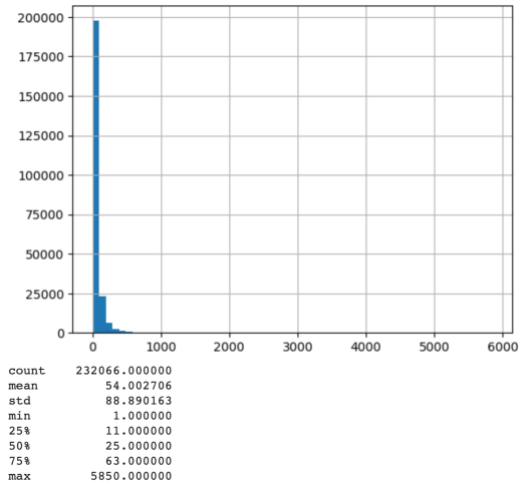

Figure 5: Histogram of Text Lengths in Dataset

Removing Posts with High Word Count: We then got the word count of the posts that had been preprocessed. The 75th percentile of 63 words is significantly different from the maximum of 5,850 words. We have chosen to employ the 75th percentile as a word count barrier, meaning that processed posts with more than 63 words are eliminated to optimize for model training in subsequent rounds. We will be able to focus on training the model on shorter postings and shorten training time by subsetting the dataset.

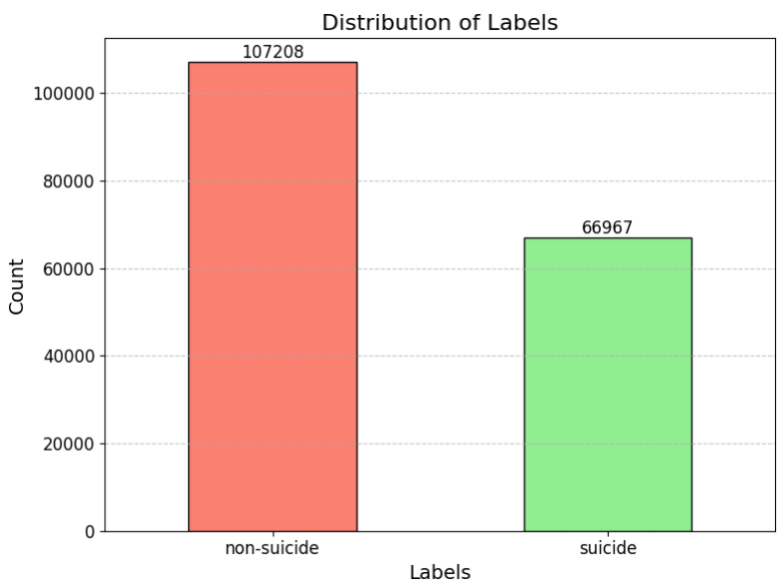

Figure 6: Class Label distribution of preprocessed dataset

The class distribution of the cleaned dataset is displayed above. Compared to the original dataset distribution. Suicidal and non-suicidal text now have a about 40:60 ratio as opposed to the 50:50

ratio in the original dataset class distribution. This observation suggests that larger word counts in suicidal posts were eliminated during data cleaning. The model's performance won't be impacted by the tiny imbalance in the class distribution, which can still be considered a typical classification issue.

The dataset is then split into the train, text, and validation sets in an 80:10:10 ratio. Further exploration of the datasets is done through visualizations.

Exploratory Data Analysis

Word Cloud

Word Cloud for Suicide Texts
know
feel
want
time
help
die
think
right
people
care
tell
work
bad
fucking
friend
Word Cloud for Non-Suicide Texts
know
want
think
feel
people
guy
right
love
girl
shit
say
fuck
day
time
friend
talk
help
good
thing
try
way
post
actually
never
person
look
start
school
work
bad
let
year
kind
hate
low
make
play
stuff
probably
cometoday
geddit
need
always
ask
sleep
nothing
find
watch
take
break
parent
lot
cool
kid
happen
hour
thank
yeah
live
fucking
see
well
tell
though
class
matter
want live
death
sure
leave
sorry
hurt
give
world
come
point
today
start
find
hate
thing
happy
feeling
commit suicide
anymore
always
mean
week
actually
live
kill
guess
nothing
let
matter
shit
love
go
hope
day
well
fucking
family
stop
wish
reason
try
understand
tired
pain
month
end
maybe
lose
long
good
suicidal thought
friend
talk
post
need
plan
happen
probably
year
suicidal
way
never
bad
fuck
feeling
sure
death
want live
school
school
work
life
life
start
find
today
point
come
hate
thing
happy
sorry
leave
hurt
give
world
need
plan
never
way
bad
year
happen
probably
suicidal
suicidal thought
good
long
friend
talk
post
start
find
today
point
come
hate
thing
happy
sorry
leave
hurt
give
world
need
plan
never
way
bad
year
happen
probably
suicidal
sure
death
want live
feeling
commit suicide
anymore
always
mean
week
actually
live
kill
guess
nothing
let
matter
shit
love
go
hope
day
well
fucking
family
stop
wish
reason
try
understand
tired
pain
month
end
maybe
lose
long
good
suicidal thought
friend
talk
post
need
plan
happen
probably
year
suicidal
way
never
bad
fuck
feeling
sure
death
want live

Average Text Length Distribution

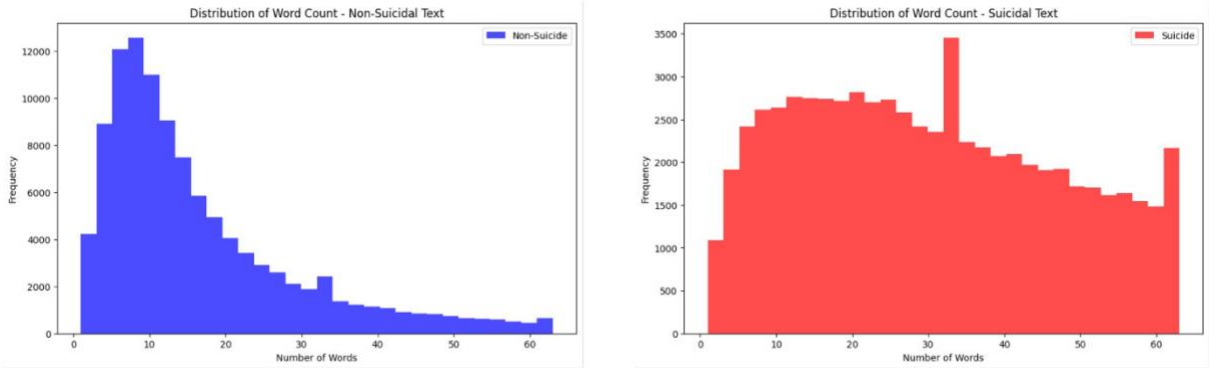

Figure 8: Distribution of Average Text Length

In Figure 8 above, we looked at the distribution of postings' average text lengths as represented by histograms. Compared to non-suicidal texts, which have a distribution that is right-skewed, suicidal texts have a more uniform distribution. Accordingly, messages that are not suicidal tend to be shorter than texts that are. This result supports our first hypothesis, which is that people who are contemplating suicide tend to express more emotions and write lengthier postings.

Frequent Bigrams

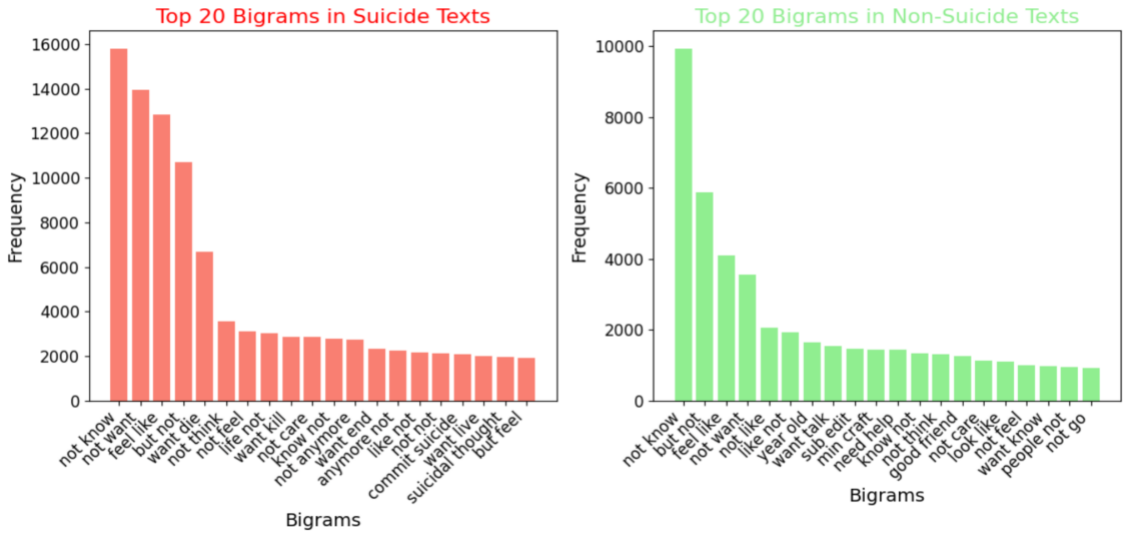

Figure 9: Most Frequent Bigrams

In a text, a bigram signifies two consecutive words. We examine the most prevalent bigrams in the Figure above, which can offer useful context for the dataset. Examples of non-suicidal bigrams are "year old" and "good friend," whereas the most common suicidal bigrams are "want die," "want kill," and "commit suicide." This shows that people who write suicide texts are frequently going through very bad things, emphasizing how vital it is to get treatment right away. We are not interested in the bigrams created from the original word sequence because the bigrams were produced after text preprocessing.

Word Embeddings

Word embeddings[4] are a crucial advancement in natural language processing (NLP) and machine learning, offering a sophisticated way to represent words as dense vectors in a high-dimensional space. This representation captures semantic relationships between words, allowing models to understand language contextually. For instance, words with similar meanings, such as "king" and "queen," are positioned closer together in the vector space, reflecting their semantic proximity.

Additionally, word embeddings enable NLP models to generalize better, facilitating tasks like document classification and clustering by leveraging the relationships captured in the embedding space. Overall, word embeddings play a vital role in bridging the gap between human language and machine understanding, driving progress in NLP applications.

Our model will have to learn the word embeddings during training from scratch if pre-trained word embeddings are not loaded into the embedding layer. This could be difficult for the reasons listed below. First off, there's a chance that a sizable portion of the training set contains uncommon terms. Due to information scarcity, learned embeddings may not accurately represent these words. Secondly, because there are a lot of trainable parameters when learning the embeddings from the beginning, our training procedure will take a lot longer.

Models such as Word2Vec, GloVe, and fastText are commonly used for deep learning model initialization due to higher effectiveness as compared to random initialization. In this section, we aim to derive representations using the Word2Vec algorithm. Custom Word2Vec embeddings will be pre-trained using the training dataset.

Word2Vec

Word2Vec[5] is a technique developed by Google in 2013, which improved natural language processing by providing a way to efficiently compute word embeddings. The technique transforms words into dense, multi-dimensional vectors, capturing their semantic relationships in a continuous vector space. Word2Vec operates on large text corpora, learning these representations through neural networks, making it highly effective for processing textual data. Its ability to encode both syntactic and semantic relationships between words has made it a foundational tool in NLP, powering tasks such as document classification, text similarity measurements, and machine translation.

The Continuous Bag-of-Words model is one of the two approaches used in Word2Vec to learn word embeddings. In CBOW, the model predicts a target word based on the context words surrounding it. The input is a set of one-hot encoded vectors representing the context words, and the model, using a simple neural network architecture, generates a probability distribution over the vocabulary.

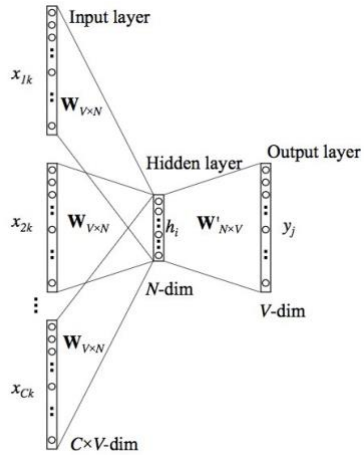

Figure 10: Architecture of a CBOW Model

CBOW's efficiency in processing large text corpora stems from its focus on predicting words from context, making it an effective method for generating dense, contextually meaningful word embeddings that encapsulate the relationships between words.

Our train dataset, which has a vocabulary size of about 20,400 and 139,500 data points, should offer a suitable learning environment for a custom-trained embedding layer.

Model Building

We'll be developing 5 models and assessing how well they categorize texts containing suicide threats. We are attempting to anticipate a binary variable in our issue statement—suicide or non-suicide.

1. Logistic Regression
2. CNN (Convolutional Neural Network)
3. LSTM (Long Short-Term Memory Neural Network)
4. BERT (Bi-directional Encoder Representations from Transformers)
5. ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately)

1. Logistic Regression

Logistic Regression[6] is a widely used statistical model, particularly effective as a baseline model for text classification tasks in natural language processing (NLP). This model operates on the principle of predicting binary or multi-class outcomes based on input features. In the context of NLP, the input features can be derived from textual data, often represented as numerical vectors, such as word embeddings or other feature extractions. Logistic Regression models the relationship between these features and the probability of a particular class label.

Model Architecture

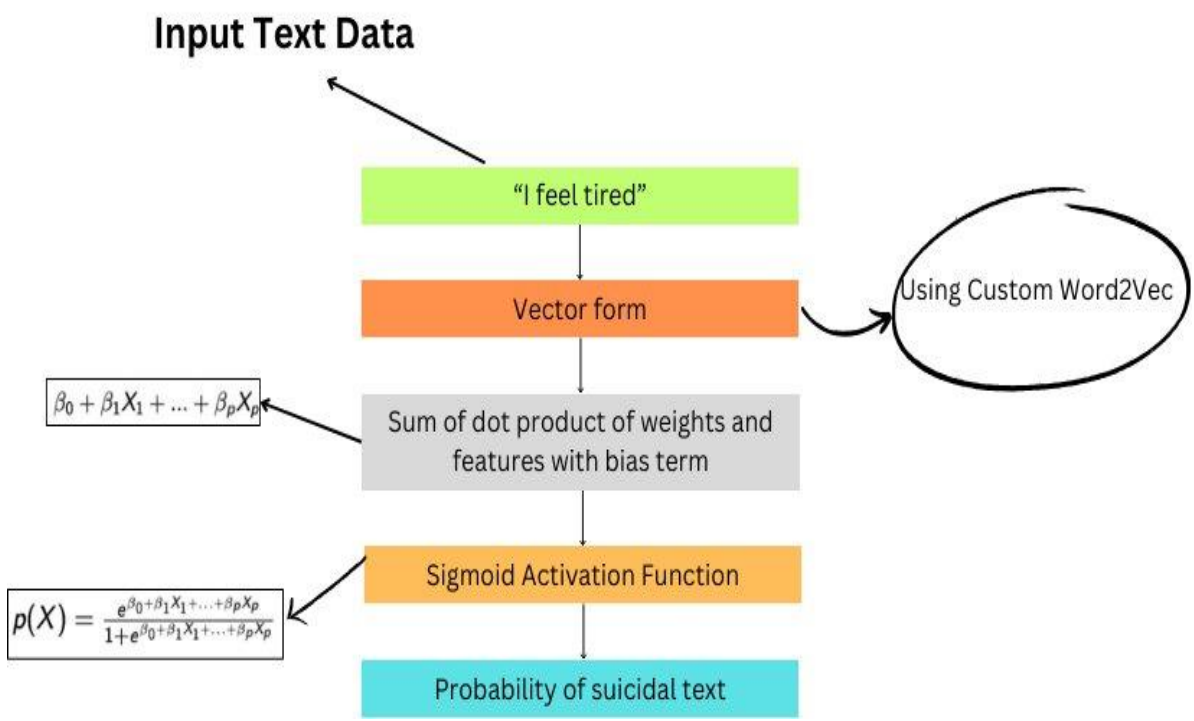

Figure 11: Architecture of Logistic Regression Model

The architecture begins with input text data which undergoes a transformation into a vector form, utilizing a custom Word2Vec model to convert words into dense numerical representations. Next, the vector form enters a linear model, where it is multiplied by a set of weights and summed along with a bias term, producing a linear combination of features. This combination is then fed into a sigmoid activation function, which maps the linear output to a probability between 0 and 1, indicating the likelihood of the text belonging to a specific class. In this context, the probability indicates the likelihood of the text being associated with suicidal content, allowing for immediate classification.

Model Performance

The performance of the baseline Logistic Regression model with the custom Word2Vec training provides a good baseline accuracy, which is the metric considered since we want to maximize the True Positives.

```
Test set accuracy 0.9054337519036809
              precision    recall  f1-score   support

           0       0.92      0.92      0.92     21414
           1       0.88      0.88      0.88     13387

    accuracy                           0.91     34801
   macro avg       0.90      0.90      0.90     34801
weighted avg       0.91      0.91      0.91     34801
```

Figure 12: Logistic Regression Metrics

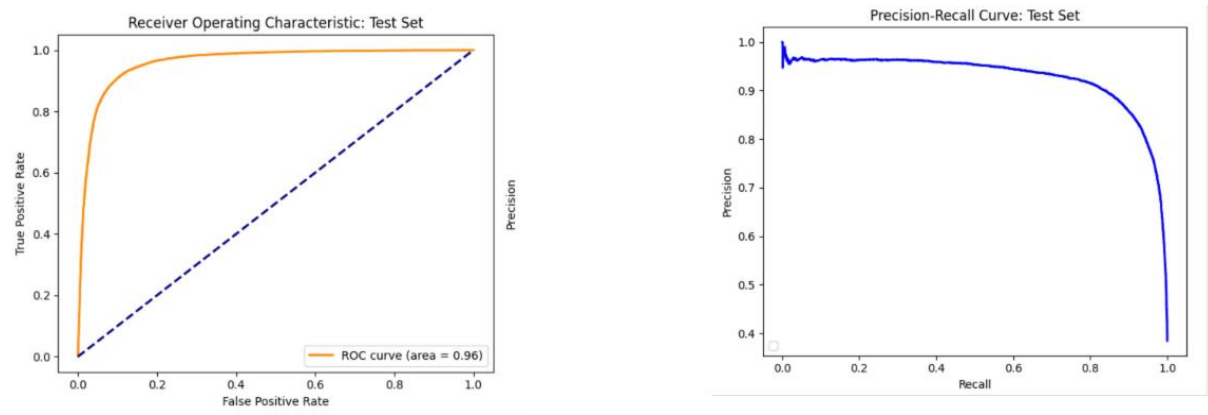

Figure 13: ROC Curve and Precision-Recall Curve

The ROC curve demonstrates the model's ability to distinguish between positive and negative classes, with an area under the curve (AUC) of 0.96, indicating high accuracy. The Precision-Recall curve showcases the model's balance between precision and recall, highlighting its efficacy in maintaining precision across various recall levels.

2. CNN (Convolutional Neural Network)

Convolutional Neural Networks (CNNs)[7] are a powerful machine learning model that has proven effective for natural language processing (NLP) text classification tasks. Originally designed for image recognition, CNNs have been adapted to NLP by treating text as a sequence of input features, such as word embeddings or n-grams. In this context, CNNs apply convolutional filters to capture local patterns, such as word combinations or n-gram relationships, that contribute to a document's meaning. By capturing these nuanced relationships, CNNs can create a robust feature representation of textual data.

Compared to Logistic Regression, CNNs offer superior performance by leveraging convolutional layers to learn hierarchical patterns and features in the text, allowing them to capture complex

semantic relationships and context more effectively, resulting in higher classification accuracy and generalization.

Model Architecture

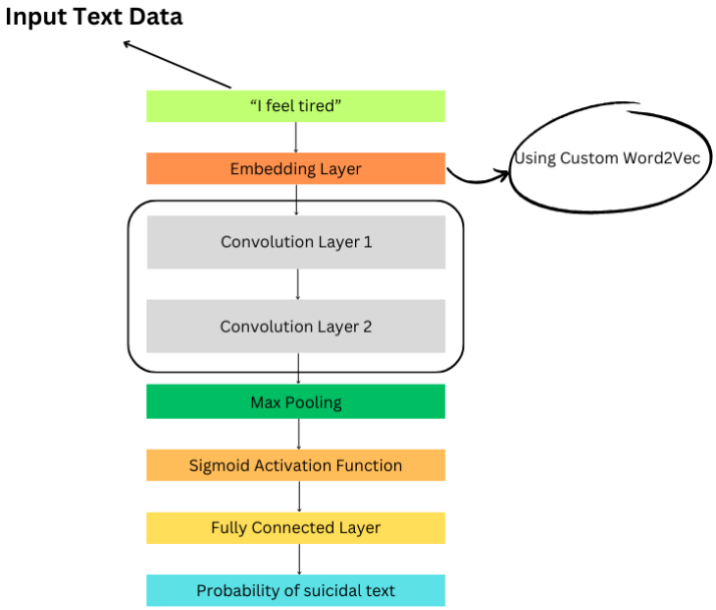

Figure 14 :Architecture of CNN Model

The input text is first transformed into a dense vector representation using a custom Word2Vec model, which is then passed to an embedding layer to generate vectorized input features. The data flows through convolutional layers, which apply convolutional filters to capture local textual patterns and relationships. These convolutional outputs are then passed through a max-pooling layer to reduce dimensionality and retain the most salient features. The pooled features are then processed by a sigmoid activation function, transforming them into a probability score between 0 and 1. Finally, a fully connected layer combines the processed features to predict the probability of the text containing suicidal content.

Hyperparameters

The first layer is an Embedding layer with an input_dim parameter representing the vocabulary size and an output_dim of 100, utilizing a predefined embedding_matrix to initialize its weights. It also has an input_length parameter, defining the length of the input sequences. Following this, a Conv1D layer with 32 filters and a kernel size of 3 applies convolutional filters across the text embeddings, extracting local features and activating them using a ReLU function. A GlobalMaxPooling1D layer then reduces the dimensionality by taking the maximum value from each feature map. The final Dense layer with a sigmoid activation function outputs a probability, classifying the text into one of two categories.

The model is compiled with the 'adam' optimizer for efficient gradient descent and uses 'binary_crossentropy' as its loss function to manage the binary classification task.

Model Performance

We compare two variations of the CNN model. One in which the weights are initialized randomly and the other with the custom Word2Vec Embeddings passed through the layers.

Model	Accuracy	Precision	Recall	F1 Score
CNN without pre trained embedding weights	0.8812	0.8366	0.8583	0.8416
CNN with Word2Vec pre trained embedding weights	0.9224	0.9059	0.8902	0.8941

Figure 15: CNN Metrics

It's observed that the model with the pre-trained Word2Vec Embeddings perform better and has a noticeable increase in accuracy.

3. LSTM (Long Short-Term Memory Neural Network)

Long Short-Term Memory (LSTM)[8] networks are a type of recurrent neural network (RNN) specifically designed to handle sequential data, making them well-suited for NLP text classification tasks. LSTM networks excel at capturing long-term dependencies in text, thanks to their unique cell architecture, which consists of an input gate, output gate, and forget gate. These gates allow LSTM networks to selectively store and retrieve information over extended time steps, maintaining context throughout the text.

Compared to CNNs, LSTMs offer an advantage in handling sequential data, allowing them to capture relationships between words across larger distances, which can be critical for accurately interpreting and classifying complex texts.

Model Architecture

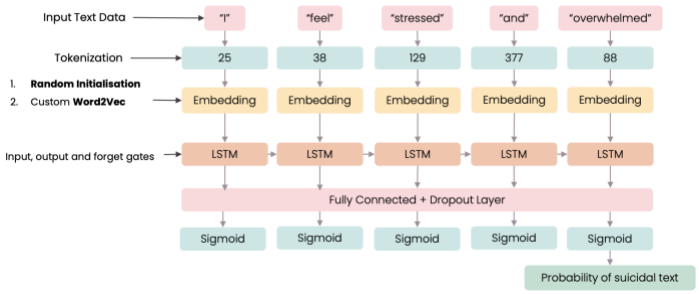

Figure 16: Architecture of LSTM Model

The process begins with input text data, which is tokenized, converting each word into its numerical representation. The tokenized words are then transformed into embeddings, using

either random initialization or a custom Word2Vec model, capturing semantic relationships. These embeddings pass through a series of LSTM layers, which utilize their input, output, and forget gates to selectively retain and process information, capturing sequential dependencies within the text. Afterward, a fully connected layer with a dropout mechanism is applied, reducing overfitting by randomly deactivating some neurons. Finally, the output goes through a sigmoid activation function, mapping the result to a probability between 0 and 1, indicating the likelihood of the text being suicidal in nature.

Hyperparameters

The initial Embedding layer has an input_dim parameter representing the vocabulary size and an output_dim of 100, with weights initialized from an embedding_matrix. The embedding layer is not trainable, maintaining static representations. The model then includes two LSTM layers, the first with 128 units and the second with 64 units. The first LSTM layer also sets return_sequences=True, allowing for sequential data to pass through to the next layer. Between these LSTM layers are Dropout layers with a dropout rate of 0.2, which helps reduce overfitting by randomly deactivating some neurons. A Dense layer with 32 units and a ReLU activation follows, introducing a fully connected layer that provides non-linearity. The final Dense layer has a single unit and a sigmoid activation function, outputting a probability between 0 and 1, indicating the classification result.

Like the CNN model, the LSTM model is compiled with the 'adam' optimizer for efficient gradient descent and 'binary_crossentropy' as the loss function to manage the binary classification task.

Model Performance

We compare two variations of the LSTM model. One in which the weights are initialized randomly and the other with the custom Word2Vec Embeddings passed through the layers.

Model	Accuracy	Precision	Recall	F1 Score
LSTM without pre trained embedding weights	0.8985	0.8748	0.8579	0.8609
LSTM with Word2Vec pre trained embedding weights	0.9234	0.9093	0.8876	0.8941

Figure 17: LSTM Metrics

It’s observed that the model with the pre-trained Word2Vec Embeddings performs better and has a noticeable increase in accuracy.

Transformers Architecture

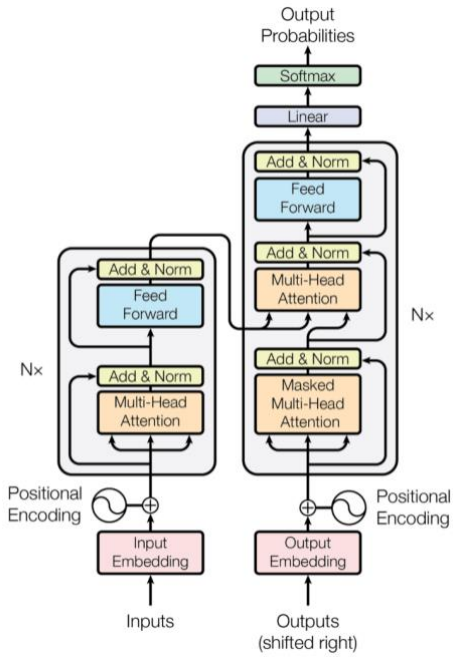

Figure 18: High-Level Transformer Architecture (Vaswani et al.)

Transformers[9] have become a revolutionary technology in the field of natural language processing (NLP). Its ability to offer a flexible and powerful framework to handle various data types, including text, images, and speech is what makes it stand out. There are two vital components to the transformer architecture: the encoder and the decoder block. While both components play an important role in the processing and generation of data, widely used powerful models such as BERT and ELECTRA which form the basis of this project are based on the transformer encoder structure. These encoder-only models have made huge advancements in text classification, sentiment analysis, and question-answering. This section solely focuses on how the encoder architecture analyses and processes the input data which is the foundational unit for BERT and ELECTRA models.

The Role of Tokenization and Embeddings in Transformers

The process of preparing texts for a transformer model starts with tokenization. Here, the text is broken down into manageable pieces such as words and subworlds. These tokens are then transformed into vectors using embedding. Tokenization is an important first step since it directly impacts how the model processes texts. By breaking down the sentence into smaller units, the model can more effectively learn and generalize the training data. Once the sentence has been tokenized, these texts are converted to vectors using embeddings. Unlike the sparse and high-dimensional vectors from one-hot encoding, embeddings are dense, with dimensions typically ranging between 256 and 1,024, even for large vocabularies[10]. This efficiency in representation does not sacrifice the richness of information; instead, embeddings provide a nuanced

understanding of language by capturing semantic similarities and relationships in the structured vector space.

Embedding: From Raw Data to Vectors

Like other machine learning models, transformers also require data to be in a numerical format-specifically vector. In the case of text processing, this can be achieved using embeddings which is a sophisticated method of representing text data in a dense vector space.

Traditionally, one-hot encoding is used to convert words into vectors. This method represents each word as a vector where one element is “hot” (set to 1) and all other elements are “cold” (set to 0). While this is straightforward to understand, this method however has a flaw when representing languages which consist of a huge number of words to make sense. Furthermore, it assumes all words are mutually independent, ignoring the rich semantic relationships between them. For instance, “movie” and “film” are conceptually similar and often used interchangeably, yet one-hot encoding would place them as far apart as possible in the vector space[10].

To solve this problem, word embedding can be used. Word embeddings represent words in a lower-dimensional, continuous vector space. This representation allows for capturing the semantic relationship between words, where the distance between words corresponds to semantic similarities or differences. Hence, embeddings can help in representing the structure of languages, where related words are placed closer to one another in the vector space and this vector space can encode meaningful patterns and relationships of the language[10].

Pre-trained Model Embeddings: Leveraging Large-Scale Language Understanding

When working with a transformer model, often pre-trained embeddings are generated from extensive corpora covering a broad spectrum of the language. Popular models like BERT (Bidirectional Encoder Representations from Transformers) and ELECTRA (Efficiently Learning an Encoder that Classifies Token Replacements Accurately) are equipped with their own embeddings, refined through pretraining on vast datasets. This pretraining fills the model with a deep, nuanced understanding of language patterns, relationships, and contexts before any task-specific training begins. This approach significantly enhances the models' ability to interpret and generate language with remarkable accuracy and depth, setting the stage for their exceptional performance on a wide range of tasks.

Positional Encoding

Positional encoding is an important component in the transformer model. This process addresses one of the limitations of text processing: the lack of a mechanism to understand the order of input tokens. Since transformers process input data in parallel rather than sequentially, they do not naturally account for the position of each token within a sequence. Positional encoding is introduced to fill the model with the ability to recognize the order of words or tokens, which is vital for many tasks in natural language processing where the meaning of a sentence can dramatically change with the order of words.

This process involves adding extra information to each token to represent its position in the sentence. The idea here is to provide the model with a sense of each token's order or position in the context of the entire sentence. This added information allows the transformer to consider the word's order, which is essential for understanding language structure and nuances.

Using Sine and Cosine Functions for Positional Encoding

The transformer model uses a specific method for positional encoding, involving sine and cosine functions. This approach was introduced in the paper "Attention is All You Need" by Vaswani et al., which presented the transformer model. The positional encoding formula uses different frequencies of sine and cosine functions to encode each word's positional information. The intuition behind using these functions is to enable the model to easily learn to attend by relative positions.

The mathematical formulation for positional encoding is as follows[9]:

$$PE_{\{(pos,2i)\}} = sin\ sin\left(\frac{pos}{10000^{\left\{\left(\frac{2i}{d_{\{model\}}}\right)\right\}}}\right)$$

$$PE_{\{(pos,2i+1)\}} = cos\ cos\left(\frac{pos}{10000^{\left\{\left(\frac{2i}{d_{\{model\}}}\right)\right\}}}\right)$$

Here,

$PE_{\{(pos,2i)\}}\ and\ PE_{\{(pos,2i+1)\}}$ refer to the positional encoding for the position pos and dimension 2i or 2i + 1 respectively

pos is the position of the token in the sequence

i is the dimension index

d_{model} is t he dimensionality of the token embeddings (usually 512 or 1024)

The sine function is applied to even indices (2i), and the cosine function is applied to odd indices (2i+1).

The application of position encoding can be seen below:

Suppose we have the sentence "Your cat is cute." To demonstrate how positional encoding works, we'll focus on the subset of the sentence "Your cat is." Each word is first converted into a 512-dimensional vector using an embedding layer. These vectors represent the semantic meaning of the words within an embedding space.

For each word in the sentence, we have a 512- dimensional vector. For each dimension i in this vector, if i is even, we apply the sine function and if i is odd, we apply the cosine function.

This process generates a positional vector for each word, encoding the word's position in the sentence. These positional vectors are then added element wise to the original embedding vectors. The resulting vectors are 512-dimensional and incorporate both the semantic meaning of the words and their positions within the sentence.

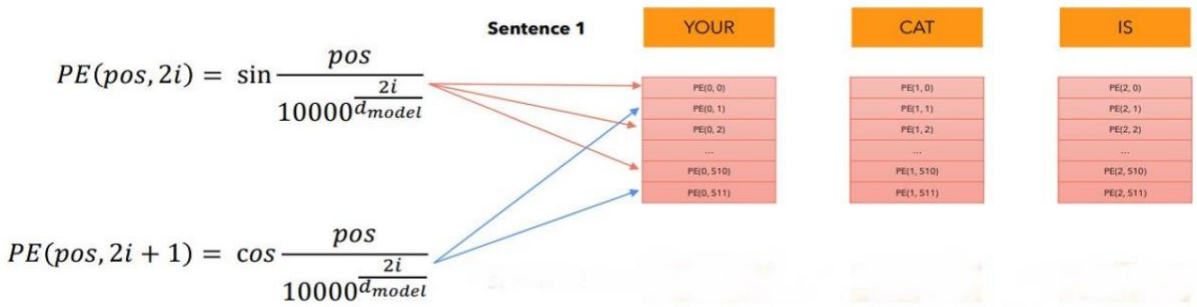

Figure 19: Positional Encoding [12]

Attention Mechanism

The key mechanism that makes the transformer architecture such a revolutionary technology is the attention mechanism. When dealing with natural languages, the words present in a sentence are processed word by word or in a sequential manner. This leads to a significant issue since languages are complex by nature with intricate grammatical structures and relationships between words. Processing each word without considering their context can obscure these crucial connections. The attention mechanism addresses this issue where each word "attends" to all other words in the sentence, determining which words in the sentence are most significant in the specific context [[11]. This idea can be highlighted with the help of an example. Given the sentence 'The train left the station on time'. Because of a human's years of training and complex understanding of natural languages, one can infer that the word 'station' refers to the 'train station' and not a space station or any other form of station. In other words, a human knows that when the word 'station' comes in the sentence, one should pay attention to the word 'train'[10]. This process can be replicated in the transformer architecture through the self-attention mechanism.

Self-Attention

The process of self-attention starts with the matrix obtained after embedding and the positional encoding process. This matrix is then linearly transformed using three weight matrices, the query weight matrix Wq, the key weight matrix Wk, and the value weight matrix Wv[11]. These matrices are adjusted through the training process. These matrices are initialized with random values and are then adjusted through the process of backward propagation which uses gradients to minimize the loss. As a result of this linear transformation, three matrices are produced – query, key, and value matrices. Using these three matrices the attention scores are calculated which signals how much attention each word in the sentence should pay to other words in the same sentence. The attention values are calculated using the formula,

$Attention(Q, K, V) = softmax\left(\frac{QK^t}{\sqrt{d_k}}\right).V$. here, Q = query matrix, K = key matrix, V = value matrix

and d = dimension of the embedded and encoded input matrix[9]. A SoftMax function is applied so that the attention score is scaled between 0-1 and the attention values added together total to 1. The matrices are divided by the square root of the dimension to stabilize the process. An example can be used to demonstrate and understand how self-attention works.

As discussed earlier, suppose there is a sentence 'The train left the station on time.' After embedding and positional encoding, we have a matrix of dimension 7 x 512. This matrix is then multiplied with the query, key, and value weight matrices achieved through the process of training. As a result of this multiplication, three matrices are produced, Query matrix (7,512), Key matrix (512,7), and Value matrix (7,512). The $softmax\left(\frac{QK^t}{\sqrt{d_k}}\right)$ part of the equation is what forms the basis of the attention matrix. We will use the images below as a reference.

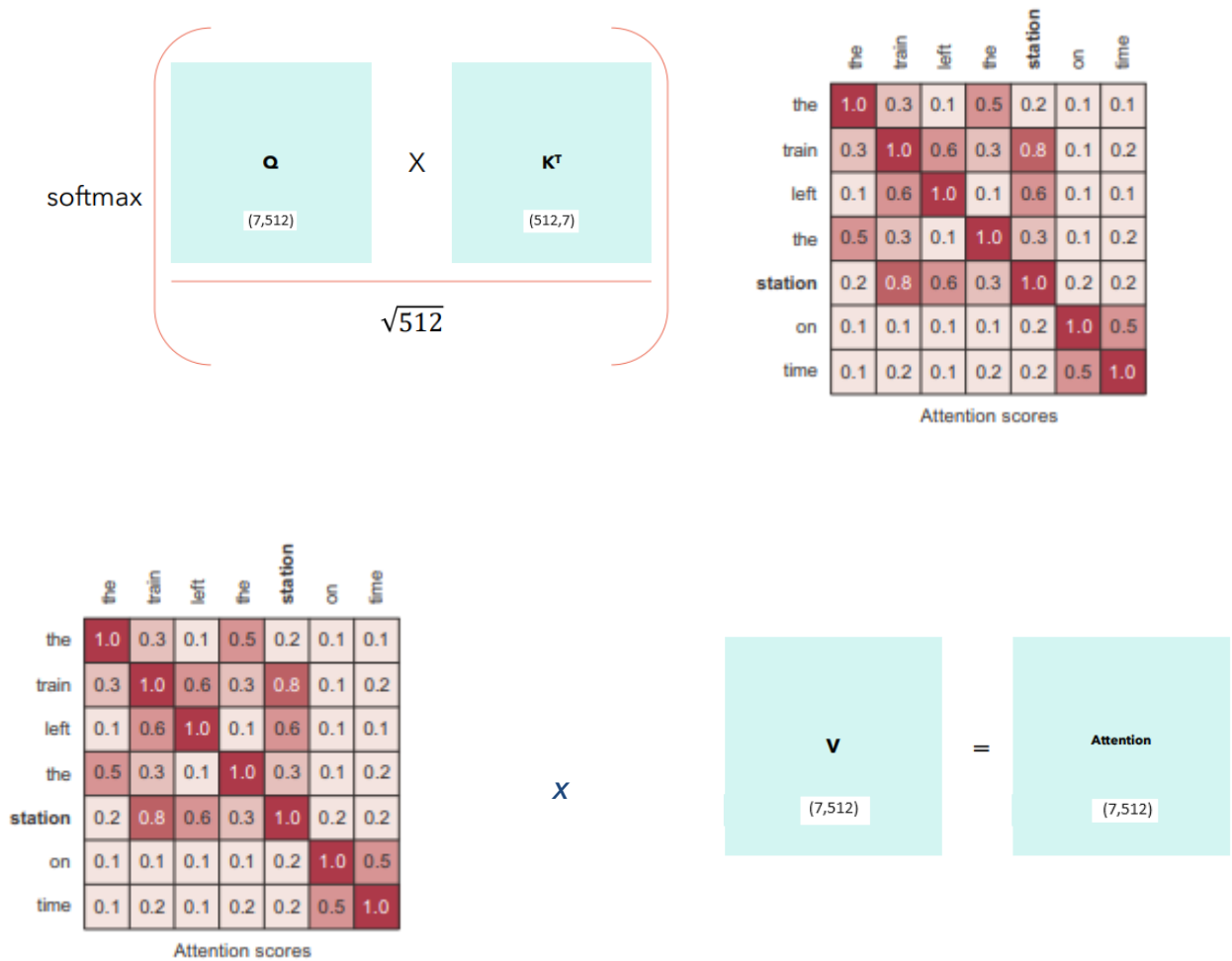

Figure 20: Attention Mechanism [10][12]

From the figure above the attention matrix is achieved after applying the SoftMax to the multiplication of query and key matrices. The attention matrix shows how much attention each word should pay to the other words in the sentence. As it is seen, the word always has the highest attention to itself. Following the discussion from earlier, the word station has the highest attention to the word train which signifies that the station in discussion implies a train station. Finally, this attention matrix is multiplied by the value matrix to get a final matrix of dimension (7,512) where each row of this matrix consists of the embedding, positional encoding, and the interaction of each word with other words in the sentence.

Multi-head Attention

As referenced in the 'Attention is all you need paper', multi-head attention is the process of splitting the query, key, and values are split into smaller chunks. The initial query, key, and value matrices are sent through n-independent set of dense projection which produces n smaller chunks of vectors. Each vector is then processed through a self-attention mechanism, called a head. Finally, the output of each of these self-attention heads is then concatenated to get a final attention matrix[10]. The figure below can be used to highlight this example.

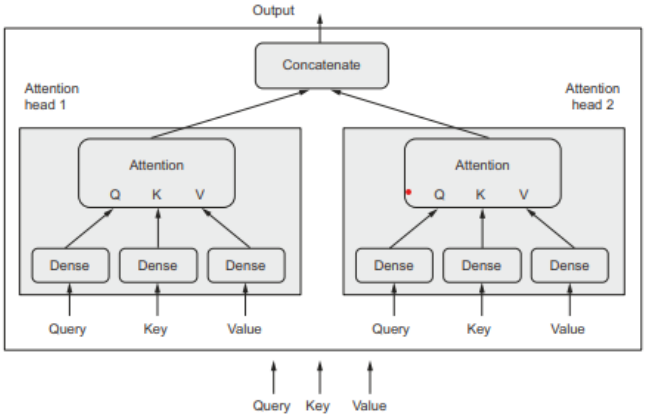

Figure 21: Architecture for Multi-Head Attention [10]

As seen in the figure above, the initial query, key, and value matrices are passed through 2 independent sets of dense projects to get two smaller chunks of vectors. These individual chunks then go through each attention head to get a final attention matrix that is concatenated. Since this process involves using more than one self-attention head, the entire process is called multi-head attention. The advantage of using multiple heads as compared to a single head is that each head learns something different as compared to just stateless numerical transformations. Furthermore, having multiple heads also provides more representational power and contextual understanding[9,10].

Residual Connection, Layer Normalization, and Feed Forward

Another crucial aspect of the transformer encoder's implementation is the use of residual or skip connections. In large model architectures like transformers, where gradients play a vital role in minimizing the loss function, the vanishing gradient problem can occur. This issue causes gradients to shrink during backpropagation, affecting the model's ability to learn effectively. Residual connections mitigate this problem by allowing gradients from earlier layers to bypass subsequent layers, preserving crucial information, and aiding the training process[13]. Furthermore, the multi-head attention outputs undergo multiple normalization steps to stabilize training and reduce convergence time[14]. This process helps manage gradient scales for smoother optimization. These normalized outputs then pass through feedforward networks with ReLU activation functions, so that the texts have a richer representation. Normalization follows this step as well.

This is the core functionality of the transformer encoder: converting natural language text into encodings that machines can process. This encoder architecture forms the basis of powerful models like BERT and ELECTRA, which are the focus of our analysis.

Transformer Decoder

A transformer decoder functions similarly to the encoder. The key difference is that the input for the decoder part consists of the output that is to be determined. For example: if the task at hand is a machine translation task from English to Spanish, the input is the translated Spanish sentences. These texts also go through the same process, but the only key difference is in the attention mechanism. Since the decoder's goal is to learn the structure in machine translation, during the training process, the attention part is masked where the output only depends on the previously processed words, not the words that are yet to be processed. This can be achieved through the process of masking the $softmax\left(\frac{QK^t}{\sqrt{d_k}}\right)'s$ output in such a way that the words that are yet to be processed, have their attention values converted to negative infinity. The SoftMax of negative infinity is then converted to 0, leading to the attention for unseen words being masked. This technique guides the decoder to focus on the translated text pattern, improving its grasp of the underlying principles of language translation[15].

The other key part of the decoder architecture is how the second multi-head attention process works. The second multi-head attention layer in the decoder is crucial. It utilizes the encoder's output to understand the source language structure (English). This attention layer then draws connections between the partially translated output (Spanish) and the source language. In translation, this allows the decoder to determine how concepts and word order in English correspond to the most appropriate expression in Spanish. At the end of the training process, the decoder understands the language translation task.

Like the transformer, the decoder also consists of residual connections to help the gradient flow seamlessly and normalization also takes place to better understand the language translation process and give this process a richer representation.

4. BERT

Bidirectional Encoder Representations from Transformers (BERT) is a state-of-the-art NLP model developed by Google, designed to handle a wide range of NLP tasks, including text classification. BERT leverages the transformer architecture, specifically its encoder segment, which processes text bi-directionally, capturing context from both directions simultaneously. This bidirectional approach enables BERT to understand nuanced relationships between words, making it highly effective for text classification tasks like sentiment analysis, spam detection, and topic classification. BERT is pre-trained on a vast corpora and fine-tuned on specific tasks, allowing it to adapt to various text processing needs with minimal additional training.

Compared to other deep learning models, BERT's bidirectional nature and pre-trained foundation allow it to capture richer context and relationships between words, resulting in superior performance in text classification tasks. Its ability to handle diverse NLP applications out of the box, along with its fine-tuning capabilities, makes it an efficient and versatile choice for NLP classification.

Model Architecture

The Masked Language Model (MLM) is a key pre-training task in the BERT (Bidirectional Encoder Representations from Transformers) model, designed to capture rich linguistic knowledge. In this task, a percentage of tokens in a given text sequence are randomly masked, and BERT's goal is to predict the masked tokens by considering the context from both directions. This bidirectional training enables BERT to understand the relationships between words and how they interact within a sentence. The MLM approach improves BERT's ability to handle a variety of NLP tasks, including text classification, by enabling it to learn robust, contextually informed word embeddings.

The Next Sentence Prediction (NSP) task is another pre-training component of BERT, focusing on learning the relationships between sentences. In this task, BERT is presented with pairs of sentences, and it must predict whether the second sentence naturally follows the first. This helps BERT understand discourse-level structures, capturing how sentences relate to each other. The NSP task enhances BERT's ability to manage text classification, document summarization, and other NLP applications by enabling it to recognize and process larger contextual relationships, improving its understanding of entire texts.

Model Performance

```
<class 'transformers.trainer_utils.EvalPrediction'>
{'test_loss': 0.1044866144657135,
 'test_accuracy': 0.97525548283385,
 'test_precision': 0.9678912783751493,
 'test_recall': 0.967746752277139,
 'test_f1': 0.9678190099305608,
 'test_runtime': 177.4048,
 'test_samples_per_second': 98.182,
 'test_steps_per_second': 12.277}
```

Figure 22: BERT Metrics

We observe a significant improvement in accuracy when we use the Fine-tuned BERT model.

5. ELECTRA

ELECTRA, an acronym for Efficiently Learning an Encoder that Classifies Token Replacements Accurately abbreviated is one of the most advanced pre-trained transformer models that was developed by Google[16]. ELECTRA is very similar to other pre-trained models like BERT but the advantage that it has over these models is that it is more efficient and outperforms these models with minimal computational power. What makes an ELECTRA model stand out compared to BERT-like models is the use of a unique pre-training process called Replaced Token Detection (RTD).

BERT-like models use a masked-language model technique where it masks certain tokens in the input text data that is passed to it. This process tends to corrupt the input text sequence while

only learning to predict just the mask tokens instead of predicting every single token. This leads to the amount of information learned from the input text missing out on complicated grammatical structures and language nuances. BERT-like models can still be used to predict every single token; however, this comes with an added computational cost[16].

ELECTRA also utilizes an encoder for processing input sequences, like BERT, but the key distinction is how it handles the masked tokens. Instead of directly predicting the masked tokens in BERT, ELECTRA uses a generator model to generate new words and replace these masked tokens. Once these masked tokens are replaced, a discriminator is used to identify which of the tokens in the replaced sequence are fake. Inspired by the generative adversarial network (GAN), the idea is to identify between real and fake tokens. This allows a more accurate representation of the text data and a better contextual understanding of the language that is being used[16]. Using the entire text sequence rather than only a subset as in BERT, allows the model to understand the structure and nature of the language that it deals with. As a result of this, ELECTRA requires less training data and computational resources compared to BERT in understanding the language passed to it. An example shown below can be used to understand how the ELECTRA models work:

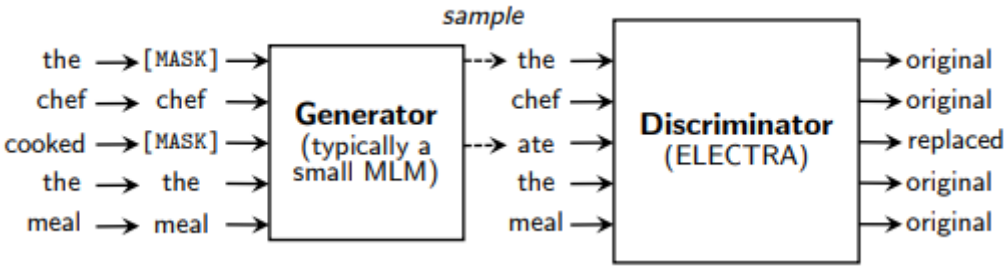

Figure 23: Generator Discriminator Flow [16]

First, an input sequence is taken and is pre-trained to perform masking. So, random words in the provided sequence are masked. As shown above, pretraining masks words – 'the', and 'cooked'. This masked sequence is then passed through a generator model to generate and replace these masked tokens. The generator model generates 'the' and 'ate' to replace the masked tokens. Finally, the new sequence goes through a discriminator model that identifies which of these tokens in the new sequence are from the input sequence and which of them are replaced. The generator tries to generate fake tokens to confuse the discriminator. By identifying these fake tokens, the discriminator understands the language structure and nuances of the language it deals with, and this is how the training occurs in an ELECTRA model.

Model Performance

In our analysis, we utilized the ELECTRA base model through the Hugging Face Transformers. The base model is taken to allow a fair comparison between the performance of ELECTRA and the BERT model. The preprocessing for the ELECTRA model was done by the tokenizer present in the hugging face model and this model was fine-tuned for one epoch to avoid overfitting with a batch size of 8.

```
<class 'transformers.trainer_utils.EvalPrediction'>
{'test_loss': 0.0993182510137558,
 'test_accuracy': 0.9761740727982546,
 'test_precision': 0.9704913121629718,
 'test_recall': 0.9674481110945199,
 'test_f1': 0.968967322216406,
 'test_runtime': 177.3651,
 'test_samples_per_second': 98.204,
 'test_steps_per_second': 12.28}
```

Figure 24: ELECTRA Metrics

RESULTS

We compile the results of all the models executed in the table below. The baseline Logistic Regression model, CNN and LSTM with the custom Word2Vec embeddings, and BERT and ELECTRA fine-tuned on the dataset.

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.9054	0.9055	0.9054	0.9055
CNN	0.9224	0.9059	0.8902	0.8941
LSTM	0.9234	0.9093	0.8876	0.8941
BERT	0.9752	0.9678	**0.9677**	0.9678
ELECTRA	**0.9761**	**0.9704**	0.9674	**0.9689**

Table 1: Comparison of 5 models

Logistic Regression achieves an accuracy of 0.9054, with comparable values across the other metrics. The CNN model shows an accuracy of 0.9224, demonstrating slightly better performance. LSTM improves further with an accuracy of 0.9234, indicating a robust handling of sequential data. BERT demonstrates a significant jump, achieving 0.9752 accuracy, along with strong recall and F1-score metrics, reflecting its capacity to capture complex semantic relationships. Finally, the ELECTRA model tops the chart with an accuracy of 0.9761 and high precision and recall values, underscoring its proficiency in NLP text classification tasks. This comparison highlights the evolution of NLP models and the growing capabilities of deep learning approaches, particularly transformer-based models, in handling text classification tasks.

(Note: Reproducing these results would require the use of Google Colab Pro. This paid service offers enhanced computational resources, including faster GPUs, longer runtimes, and more RAM, which are essential for training and fine-tuning the models described.)

CONCLUSION

Youth depression in the USA is still a major social problem, and prompt intervention depends on early detection. Our project has successfully identified suicidal wording in social media posts, and we have developed models that have produced the best accuracy of 97.61% using ELECTRA.

Looking ahead, the immediate goal is to integrate an advanced chatbot powered by a Large Language Model (LLM) into our system. This chatbot will serve as a direct interface for individuals in distress, offering immediate, sensitive responses to their concerns. The LLM, trained on billions of parameters and a large corpus, will provide nuanced, contextual awareness, capturing the subtleties of text to ensure appropriate and empathetic engagement. Fine-tuning this LLM with domain-specific data on mental health will tailor its responses to be sensitive to the context of suicidal ideation, ensuring it provides accurate information and comfort.

This integration not only strengthens the immediate support system for those in need but also scales our detection capabilities significantly. By automatically monitoring online communications, the chatbot can alert professionals or provide resources directly to at-risk individuals, addressing the critical time window for intervention. Furthermore, continuous learning from user interactions and feedback will refine the chatbot's responses, enhancing its ability to address diverse user needs.

In parallel, ongoing work will focus on improving the underlying models, including exploring newer architectures for greater accuracy and efficiency in text classification. Collaborations with mental health professionals will ensure ethical considerations and sensitivity are at the forefront, further guiding the project's development.

REFERENCES

[1] “Trends in U.S. Depression Prevalence From 2015 to 2020: The Widening Treatment Gap.” 2022. NCBI. https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9483000/.

[2] https://www.kaggle.com/datasets/nikhileswarkomati/suicide-watch/

[3] E. P. P. Mon, Y. K. Thu, T. T. Yu and A. Wai Oo, "SymSpell4Burmese: Symmetric Delete Spelling Correction Algorithm (SymSpell) for Burmese Spelling Checking," *2021 16th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*, Ayutthaya, Thailand, 2021, pp. 1-6, doi: 10.1109/iSAI-NLP54397.2021.9678171.

[4] Yoshua Bengio, Réjean Ducharme, Pascal Vincent, and Christian Janvin. 2003. A neural probabilistic language model. J. Mach. Learn. Res. 3, null (3/1/2003), 1137–1155.

[5] Mikolov, T., Chen, K., Corrado, G.S., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *International Conference on Learning Representations*.

[6] Scikit-learn: Machine Learning in Python, Pedregosa et al., JMLR 12, pp. 2825-2830, 2011.

[7] O’Shea, K., & Nash, R. (2015). An Introduction to Convolutional Neural Networks. *ArXiv, abs/1511.08458*.

[8] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. Neural Comput. 9, 8 (November 15, 1997), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

[9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17). Curran Associates Inc., Red Hook, NY, USA, 6000–6010.

[10] Chollet, Francois. Deep Learning with Python. Manning Publications, 2017.

[11] Raschka, S. (2024, January 14). Understanding and Coding Self-Attention, Multi-Head Attention, Cross-Attention, and Causal-Attention in LLMs. Ahead of AI https://magazine.sebastianraschka.com/p/understanding-and-coding-self-attention

[12] hkproj/transformer-from-scratch-notes: Notes about "Attention is all you need" video (https://www.youtube.com/watch?v=bCz4OMemCcA). (n.d.). GitHub. https://github.com/hkproj/transformer-from-scratch-notes

[13] Takase, S., Kiyono, S., Kobayashi, S., & Suzuki, J. (2022). On Layer Normalizations and Residual Connections in Transformers. ArXiv, abs/2206.00330.

[14] Xiong, Ruibin et al. “On Layer Normalization in the Transformer Architecture.” ArXiv abs/2002.04745 (2020): n. pag.

[15] Illustrated Guide to Transformers Neural Network: A step by step explanation. (2020, April 28). YouTube https://www.youtube.com/watch?v=4Bdc55j80l8

[16] Clark, K., Luong, M.-T., Le, Q. V. & Manning, C. D. (2020). ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators. International Conference on Learning Representations, .

The code for the project can be accessed at the following GitHub Repository:
https://github.com/gjoshi22/Suicide_reddit_data